

Design and Analysis of Algorithms (DAA)

Introduction to Algorithm, Space and Time Complexity

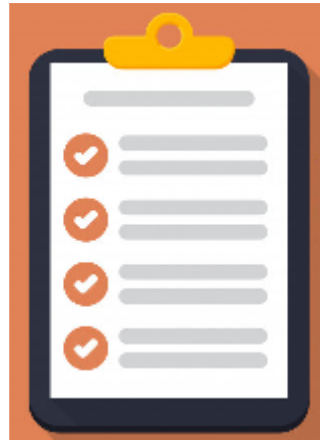
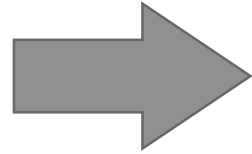
Jaina Patel
Assistant Professor
Computer Engineering Department
Dharmsinh Desai University

Overview of an Algorithm?

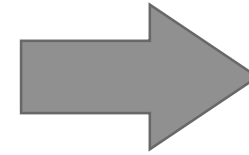
- ▶ A step-by-step procedure, to solve the different kinds of problems.
- ▶ Suppose, we want to make a Chocolate Cake.



▶ **Input**
Ingredients



Process
Recipe

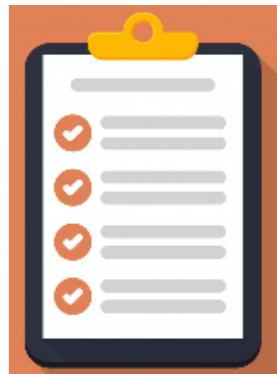


Output
Cake

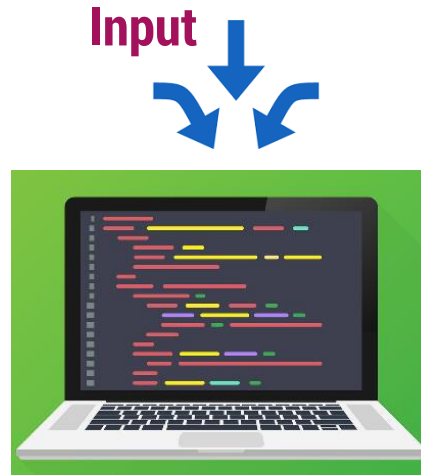
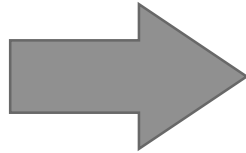
- ▶ An **unambiguous sequence of computational steps** that transform the input into the output.

Overview of an Algorithm?

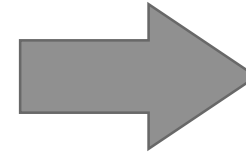
- ▶ Finite set of steps to solve a problem is called algorithm.
- ▶ A process or a set of rules to be followed to achieve desired output, especially by a computer.



Algorithm



Program



Output

- ▶ An algorithm is **any well-defined computational procedure** that takes some value, or a set of values as input and produces some value, or a set of values as output.
- ▶ Analysis is process of comparing two algorithms w.r.t. time and space.

What is an algorithm?

- ▶ *An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task.*
- ▶ Algorithm is not the complete code or program, it is just the core logic(solution) of a problem
- ▶ It can be expressed either as an informal high level description as **pseudo code** or using a **flowchart**.

Characteristics of An Algorithm

- ▶ **Input-** There should be 0 or more inputs supplied externally to the algorithm.
- ▶ **Output-** An algorithm must have **at least one** desirable output.
- ▶ **Definiteness-** Every step of the algorithm should be **clear** (unambiguous) and **precisely defined**.
- ▶ **Finiteness-** An algorithm must always terminate after a **finite number of steps**.
- ▶ **Effectiveness-** Every instruction **must be sufficiently basic** so that it can be carried out by a person using only pencil and paper.

Four Steps in Problem Solving

1. Design an Algorithm

→ Creating an algorithm by using various design techniques.

2. Validate an Algorithm

→ To assure that an algorithm computer correct answer for all possible valid inputs.

3. Analyse an Algorithm

→ Determine how much (resources) computing time and storage an algorithm requires.

4. Test a program

→ Executing a correct program on sample data sets and measuring the time and space it takes to compute the result.

► In this subject we will focus on **Design** and **Analysis** of an algorithm

Design of an Algorithm

- ▶ It is an art which may never be fully automated.
- ▶ There are **number of design techniques**.
- ▶ The techniques, which will be discussed in this subject are:
 - **Divide and Conquer**
 - **Greedy Technique**
 - **Dynamic Programming**
 - **Backtracking**
 - **Branch and Bound**

Analysis of an Algorithm

- ▶ Analysis of algorithms or performance analysis refers to the task of **determining how much computing time and storage an algorithm requires**.
- ▶ Analysis provides **theoretical estimation** for the required resources of an algorithm to solve a specific computational problem.
- ▶ It allows to make **quantitative comparison** of performance of different algorithms.
- ▶ It allows to predict **performance** of an algorithm **under different situations** like best case, worst case and average case.

Performance Analysis (Complexity)

▶ Space Complexity:

→ It is the amount of memory an algorithm needs to run to completion

▶ Time Complexity:

→ It is the amount of computer time an algorithm needs to run to completion

Space Complexity

- ▶ The space needed by an algorithm is the sum of **Fixed** space and **Variable** space
- ▶ **Fixed space: c**
 - ↳ Independent of the number and size of I/O
 - ↳ Includes the space for instructions, variables, and constants
- ▶ **Variable space: $S_P(I)$**
 - ↳ Depends on Instance characteristics, dependent on particular problem instance
 - ↳ Includes dynamic allocation, functions' recursion
- ▶ Total space of any Algorithm/program
$$S(P) = c + S_P(\text{Instance}) \quad // \text{where } c \text{ is constant}$$
- ▶ We focus on estimating $S_P(\text{Instance})$

Space Complexity

► Example 1:

```
Algorithm abc(a, b, c)  
{  
    return  $a + b + b * c + (a + b - c) / (a + b) + 4.0$ ;  
}
```

$$S_{abc}(I) = 0$$

- Assume that one word is adequate to store values of each of *a*, *b*, *c* and result. So space needed by *a*, *b*, *c* is independent of instance characteristics.

Space Complexity

- ▶ Example 2: // Iterative function for sum

```
Algorithm Sum( $a, n$ )  
{  
     $s := 0.0$ ;  
    for  $i := 1$  to  $n$  do  
         $s := s + a[i]$ ;  
    return  $s$ ;  
}
```

$$S_{\text{Sum}}(n) \geq (n + 3) \text{ (} n \text{ for } a[], \text{ one each for } n, i, \text{ and } s)$$

- ▶ Here, instance characteristic is n . Space needed by array $a[]$ is at least n words. Depends on type of variables of array.
- ▶ One word for each variable n, i, s (i.e. total 3).

Space Complexity

- ▶ Example 3: // Recursive function for sum

```
Algorithm RSum( $a, n$ )  
{  
    if ( $n \leq 0$ ) then return 0.0;  
    else return RSum( $a, n - 1$ ) +  $a[n]$ ;  
}
```

The recursion stack space needed $\geq 3(n+1)$

- ▶ Here, instance characteristic is n .
- ▶ Total $n+1$ recursive calls. (i.e. $n, n-1, n-2, \dots, 0$)
- ▶ Each call to RSum requires at least 3 words, including space for n , *return address* and *pointer to $a[]$* .

Time Complexity

- ▶ The total time = Compile Time + Run Time

$$T(P) = c + T_P(\text{Instance})$$

- ▶ Compile time does not depend on instance characteristics.
- ▶ A compiled program will be run several times without recompilation.
- ▶ Consequently, we concern ourselves with just the run time of a program i.e. $T_P(\text{Instance})$
- ▶ **Program Step**: A program step is a syntactically or semantically meaningful program segment whose execution time is independent of the instance characteristics

Methods to compute the step count

▶ Introduce variable *count* into programs

- It is a global variable with initial value 0
- Include statements to increment *count*
- Each time a statement in the original program is executed, *count* variable is incremented by the step count of that statement.

▶ **Tabular** method

- Determine the total number of steps contributed by each statement
 $\text{steps_per_statement} * \text{frequency_for_that_statement}$
- Add up the contribution of all statements

Using *count* variable

► Example 1:

```
Algorithm Sum(a, n)
{
    s := 0.0;
    count := count + 1; // count is global; it is initially zero.
    for i := 1 to n do
    {
        count := count + 1; // For for
        s := s + a[i]; count := count + 1; // For assignment
    }
    count := count + 1; // For last time of for
    count := count + 1; // For the return
    return s;
}
```

2n+3 Steps

- Inside *for loop*, value of *count* will increase by total of 2n.
- Outside for loop, *count* will increase by 3.

Using *count* variable

- ▶ Example 2: (Simplified version of Example 1)

```
Algorithm Sum(a, n)  
{  
    for i := 1 to n do count := count + 2;  
    count := count + 3;  
}
```

2n+3 Steps

- ▶ Since we are interested in determining only the change in the value of *count*, Algorithm in Example 1 may be simplified as Example 2.

Using *count* variable

► Example 3:

```
Algorithm RSum(a, n)
{
    count := count + 1; // For the if conditional
    if (n ≤ 0) then
    {
        count := count + 1; // For the return
        return 0.0;
    }
    else
    {
        count := count + 1; // For the addition, function
                             // invocation and return
        return RSum(a, n - 1) + a[n];
    }
}
```

$$t_{\text{RSum}}(n) = \begin{cases} 2 & \text{if } n = 0 \\ 2 + t_{\text{RSum}}(n - 1) & \text{if } n > 0 \end{cases}$$

- Let $t_{\text{RSum}}(n)$ be the increase in the value of *count* when Algorithm terminates.
- $t_{\text{RSum}}(0) = 2$. When $n > 0$, *count* increases by 2 plus additional increase in else part $t_{\text{RSum}}(n-1)$

Using *count* variable

- ▶ Example 3: continue..

$$t_{\text{RSum}}(n) = \begin{cases} 2 & \text{if } n = 0 \\ 2 + t_{\text{RSum}}(n - 1) & \text{if } n > 0 \end{cases}$$

-----> Recurrence Relation

- ▶ To solve, make repeated substitution of Right side function t_{RSum}

$$\begin{aligned} t_{\text{RSum}}(n) &= 2 + t_{\text{RSum}}(n - 1) \\ &= 2 + 2 + t_{\text{RSum}}(n - 2) \\ &= 2(2) + t_{\text{RSum}}(n - 2) \\ &\vdots \\ &= n(2) + t_{\text{RSum}}(0) \\ &= 2n + 2, \end{aligned} \quad n \geq 0$$

2n + 2 steps

Tabular Method

► Example 1:

Statement	s/e	frequency	total steps
1 Algorithm Sum(a, n)	0	—	0
2 {	0	—	0
3 $s := 0.0;$	1	1	1
4 for $i := 1$ to n do	1	$n + 1$	$n + 1$
5 $s := s + a[i];$	1	n	n
6 return $s;$	1	1	1
7 }	0	—	0
Total			$2n + 3$

- Note that the frequency of the **for** statement is $n + 1$ and not n .
- This is so because i has to be incremented to $n + 1$ before the for loop can terminate.

Tabular Method

► Example 2:

Statement	s/e	frequency		total steps	
		$n = 0$	$n > 0$	$n = 0$	$n > 0$
1 Algorithm RSum(a, n)	0	—	—	0	0
2 {					
3 if ($n \leq 0$) then	1	1	1	1	1
4 return 0.0;	1	1	0	1	0
5 else return					
6 RSum($a, n - 1$) + $a[n]$;	$1 + x$	0	1	0	$1 + x$
7 }	0	—	—	0	0
Total				2	$2 + x$

$$x = t_{\text{RSum}}(n - 1)$$

- Notice that under the s/e (steps per execution) column, the **else** clause has been given a count of $1 + t_{\text{RSum}}(n-1)$. This is the total cost of this line each time it is executed.

Exercise 1

Homework

- Find the step counts for the following algorithm of “Matrix addition” using both methods

```
Algorithm Add( $a, b, c, m, n$ )  
{  
    for  $i := 1$  to  $m$  do  
        for  $j := 1$  to  $n$  do  
             $c[i, j] := a[i, j] + b[i, j];$   
}
```

Exercise 1 Solution

► Using **count** variable

```

Algorithm Add( $a, b, c, m, n$ )
{
    for  $i := 1$  to  $m$  do
    {
         $count := count + 1$ ; // For ‘for  $i$ ’
        for  $j := 1$  to  $n$  do
        {
             $count := count + 1$ ; // For ‘for  $j$ ’
             $c[i, j] := a[i, j] + b[i, j]$ ;
             $count := count + 1$ ; // For the assignment
        }
         $count := count + 1$ ; // For loop initialization and
                           // last time of ‘for  $j$ ’
    }
     $count := count + 1$ ; // For loop initialization and
                           // last time of ‘for  $i$ ’
}

```

Exercise 1 Solution

► Using **Tabular** method

Statement	s/e	frequency	total steps
1 Algorithm Add(a, b, c, m, n)	0	—	0
2 {	0	—	0
3 for $i := 1$ to m do	1	$m + 1$	$m + 1$
4 for $j := 1$ to n do	1	$m(n + 1)$	$mn + m$
5 $c[i, j] := a[i, j] + b[i, j];$	1	mn	mn
6 }	0	—	0
Total			$2mn + 2m + 1$

Time Complexity Summary

- ▶ The time complexity of an algorithm is given by the **number of steps taken by the algorithm** to compute the function it was written for.
- ▶ The number of steps is itself a function of the instance characteristics.
- ▶ Although any specific instance may have several characteristics (e.g., the number of inputs, the number of outputs, the magnitudes of the inputs and outputs).
- ▶ The number of steps is computed as a function of some subset of these.

Time Complexity Summary

- ▶ For many algorithms, the time complexity is not dependent solely on the number of inputs or outputs or some other easily specified characteristic.
- ▶ For example, the searching algorithm.
 - ➔ It may terminate in one step if x is the first element examined by algorithm, or it may take two steps (this happens if x is the second element examined), and so on.
- ▶ In other words, **knowing n alone is not enough** to estimate the run time of the algorithm.

3 kinds of step counts

- ▶ When the chosen parameters are not adequate to determine the step count uniquely, we can define 3 kinds of step counts: **best case**, **worst case** and **average**.
- ▶ The **best-case** step count is the **minimum** number of steps that can be executed for the given parameters.
- ▶ The **worst-case** step count is the **maximum** number of steps that can be executed for the given parameters.
- ▶ The **average** step count is the **average** number of steps executed on instances with the given parameters.

Difficulty with Step Counts

- ▶ Determining the exact step count (best case, worst case, or average) of an algorithm can prove to be an exceedingly difficult task.
- ▶ It is not worthy to put immense efforts to determine the step count exactly, since the notion of a step is itself inexact.
- ▶ E.g. Both the instructions
 1. $x := y;$
 2. $x := y + z + (x/y) + (x * y * z - x/z);$count as one step

Asymptotic Notation

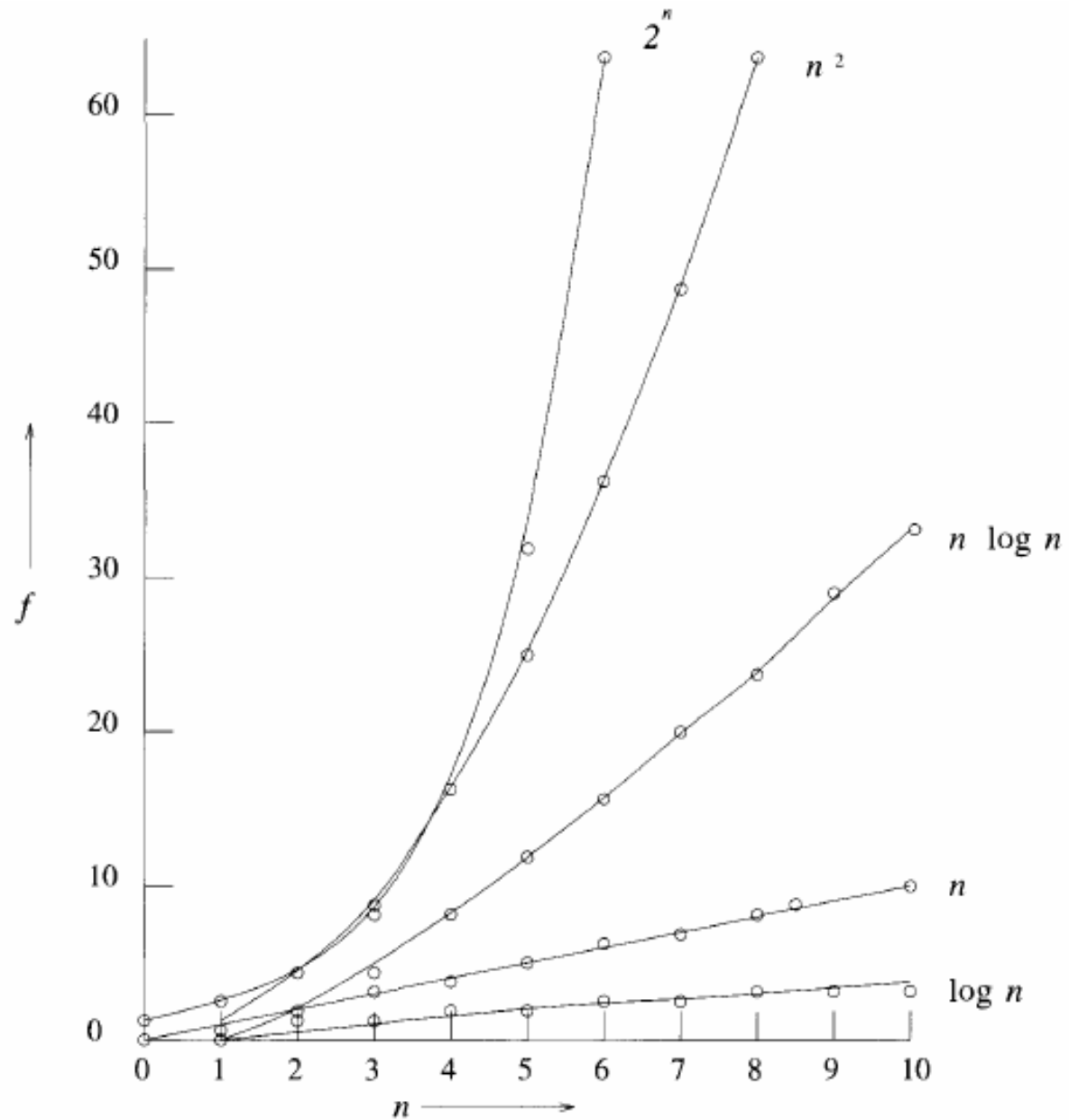
- ▶ As it is very difficult to determine exact step count, Asymptotic Notations are used to measure complexities.
- ▶ It enables to make meaningful (but inexact) statements about the time and space complexities of an algorithm
- ▶ Asymptotic Notations describe complexity in terms of some function of instance characteristic (generally denoted with n , *input size*)
- ▶ **Running time** of an algorithm is expressed as a **function of input size (n)**.
- ▶ For examples: n , $\log(n)$, $n \cdot \log(n)$, n^2 , n^3 , 2^n .

Function Values

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4,096	65,536
5	32	160	1,024	32,768	4,294,967,296

Order of Growth indicates how quickly or slowly, Running time of an algorithm grows w.r.t input size.

Plot of Function Values



References

- ▶ 1. A Book – “Fundamentals of Computer Algorithms” by Horowitz, Sahni, Rajasekaran